Họ và tên: Trương Chí Long
MSSV / Lớp: 3123410198 / DCT1234

TÌM KIẾM, CSP VÀ GAME TRONG AI

Bài 1:

a)

Tầng 2 (MAX):

- $V(A_1) = \max(3, 0) = 3$
- $V(A_2) = \max(-1, 1) = 1$
- $V(A_3) = \max(4, 6) = 6$
- $V(A_4) = \max(10, -1) = 10$
- $V(A_5) = \max(0, -2) = 0$
- $V(A_6) = \max(12, -3) = 12$
- $V(A_7) = \max(-4, -2) = -2$
- $V(A_8) = \max(-2, 10) = 10$
- $V(A_9) = \max(-5, -7) = -5$

Tầng 1 (MIN):

- $V(M_1) = \min(V(A_1), V(A_2), V(A_3)) = \min(3, 1, 6) = 1$
- $V(M_2) = \min(V(A_4), V(A_5), V(A_6)) = \min(10, 0, 12) = 0$
- $V(M_3) = \min(V(A_7), V(A_8), V(A_9)) = \min(-2, 10, -5) = -5$

Tầng 0 (Gốc MAX R):

- $V(R) = \max(V(M_1), V(M_2), V(M_3)) = \max(1, 0, -5) = 1$

b)

Nhánh tối ưu: Nút gốc MAX chọn đi vào nhánh con đầu tiên (nhánh bên trái, dẫn tới M_1).

Giá trị trò chơi (Game value): $V^* = 1$.

d)

Các nút lá không cần thăm (bị tỉa):

- Lá 6 (con của A_3)
- Lá $12, -3$ (con của A_6)
- Lá $-2, 10$ (con của A_8)

- Lá $-5, -7$ (con của A_9)
- Tổng cộng có 7 nút lá bị cắt tỉa.

So sánh:

- Thuật toán Minimax cơ bản: Đánh giá toàn bộ 18 nút lá.
- Thuật toán Alpha-Beta Pruning: Chỉ cần thăm 11 nút lá (tiết kiệm khoảng 38.9% số nút lá cần duyệt).

Bài 2:

a) Trường hợp 1: Cấu trúc tầng $A(MIN) \rightarrow B(MAX) \rightarrow C(CHANCE) \rightarrow Lá$
Nút được viền đậm là lá thứ nhất của nút C bên phải, thuộc nhánh B bên phải.
Kết luận: không thể cắt tỉa.

b) Trường hợp 2: Cấu trúc tầng $A(MIN) \rightarrow C(CHANCE) \rightarrow B(MAX) \rightarrow Lá$
Nút được viền đậm là lá thứ hai của nút B thứ 2 (bên phải) của cây con C bên phải.
Kết luận: có thể cắt tỉa.

c) Trường hợp 3: Cấu trúc tầng $A(MIN) \rightarrow C(CHANCE) \rightarrow B(MAX) \rightarrow Lá$
Nút được viền đậm là lá thứ nhất của nút B thứ nhất (nhánh trái) của C bên phải.
Kết luận: không thể cắt tỉa.

d) Sự khác nhau giữa cắt tỉa trong cây có nút ngẫu nhiên và Alpha-Beta trong cây Minimax chuẩn

- Tính đơn điệu và tính chặn (Monotonicity vs Averaging):
  + Trong Minimax: Giá trị của nút MIN là hàm không tăng (càng xét thêm con, giá trị chỉ có thể giữ nguyên hoặc giảm xuống); giá trị của MAX là hàm không giảm (chỉ có thể giữ nguyên hoặc tăng lên). Do đó, chỉ cần một con vi phạm ngưỡng ($\alpha \geq \beta$) là lập tức cắt được toàn bộ các con còn lại.
  + Trong Expectiminimax: Nút Chance tính tổ hợp tuyến tính lồi (trung bình có trọng số). Một giá trị cực nhỏ hoặc cực lớn ở một lá sau hoàn toàn có thể kéo tụt hoặc đẩy vọt giá trị kỳ vọng lên.
- Yêu cầu về miền giá trị (Bounded Utilities):
  + Alpha-Beta chuẩn không đòi hỏi cận trên/cận dưới của tiện ích trạng thái (vẫn chạy đúng với miền $(-\infty, +\infty)$).
  + Cắt tỉa trong Expectiminimax muốn hiệu quả trên diện rộng bắt buộc phải biết trước chặn trên $V_{\max}$ và chặn dưới $V_{\min}$ của miền tiện ích để ước lượng cận của kỳ vọng khi chưa duyệt hết các nhánh.

Bài 3:

a) Đúng. Giải thích: Trong trò chơi Minimax gốc, đối thủ MIN luôn chơi phòng thủ tối ưu tuyệt đối để đè giá trị của MAX xuống mức thấp nhất có thể ($V_M = \max_a \min_b U(a, b)$). Khi thay nút MIN bằng nút ngẫu nhiên (chơi theo phân phối xác suất ngẫu nhiên), đối thủ không

còn đảm bảo là đi nước cờ tồi tệ nhất cho ta nữa. Do đó, giá trị tốt nhất mà MAX đạt được trước một môi trường ngẫu nhiên V_E chắc chắn luôn lớn hơn hoặc bằng giá trị khi đối đầu với đối thủ khắc nghiệt tối ưu: $V_E \geq V_M$.

b) Đúng. Giải thích: Chính sách Minimax là chính sách an toàn bảo thủ (worst-case guarantee). Nó đảm bảo rằng với bất kỳ hành động nào của tầng dưới (dù đối thủ chọn có chủ đích hay do xúc xắc gieo ngẫu nhiên theo xác suất nào đi nữa), tiện ích thu về trong mọi kịch bản đều không nhỏ hơn giá trị minimax V_M. Do đó, kỳ vọng (tổng xác suất các kết quả) cũng chắc chắn $\geq V_M$.

c) Sai. Giải thích: V_E là giá trị kỳ vọng tối ưu chỉ đạt được khi người chơi sử dụng chính sách Expectimax tối ưu (tận dụng tối đa xác suất của môi trường). Chiến lược Minimax quá bi quan, có thể chọn một nhánh "an toàn nhưng điểm kỳ vọng thấp" để tránh rủi ro ở trường hợp xấu nhất, do đó lợi ích kỳ vọng thực tế đạt được khi dùng chiến lược Minimax có thể nhỏ hơn V_E.

d)
Expectimax với tiện ích trạng thái cuối +1/−1
Value Iteration với phần thưởng 0 ở trạng thái trung gian và +1/−1 ở kết quả thắng/thua
Giải thích:

- Minimax không phù hợp: Vì đây là trò chơi 1 người trong môi trường ngẫu nhiên (nondeterministic/stochastic), không có đối thủ thông minh đối kháng có chủ đích; Minimax coi tự nhiên là đối thủ độc hại nhất (worst-case), dẫn đến hành vi siêu bảo thủ và phi lý trí.
- Expectimax phù hợp: Vì môi trường có kết quả hành động ngẫu nhiên, mô hình bằng cây tìm kiếm gồm nút MAX xen kẽ nút Chance là chuẩn xác nhất để tối đa hóa xác suất chiến thắng (tiện ích kỳ vọng). Giả định: Trò chơi có độ sâu hữu hạn hoặc không có chu trình vô hạn (finite horizon / acyclic).
- Value Iteration phù hợp: Môi trường hoàn toàn thỏa mãn mô hình Quá trình quyết định Markov (MDP) với không gian trạng thái, xác suất chuyển $P(s'|s,a)$ và hàm thưởng $R(s) = 0$ tại các bước đi trung gian, $R(s_{terminal}) = \pm 1$. Giả định: Không gian trạng thái là rời rạc, quá trình thỏa mãn tính chất Markov.

Bài 4:
a)
Tập đỉnh $V = \{A, B, C, D, E, F\}$.
Tập cạnh $E = \{(A,B), (A,C), (A,D), (B,C), (B,E), (C,F)\}$.
Đồ thị gồm:

- Một chu trình tam giác giữa 3 đỉnh A, B, C.
- Đỉnh D nối trực tiếp với A.
- Đỉnh E nối trực tiếp với B.

- Đỉnh F nối trực tiếp với C.

b) Miền giá trị của $\{B, C, D\}$ có thể bị thay đổi.

c) Miền giá trị của $\{C, E\}$ có thể bị thay đổi.

d) Miền giá trị của tất cả các biến chưa gán $\{B, C, D, E, F\}$ đều có thể bị thay đổi.

e) Miền giá trị của $\{C, D, E, F\}$ đều có thể tiếp tục bị thay đổi.

f) Số lần backtrack xấu nhất: 0 lần.
Giải thích: Ban đầu AC được thực hiện trên toàn đồ thị hình cây. Khi cây đã Arc Consistent, việc gán giá trị bắt đầu từ gốc A rồi tỏa dần ra các nút con theo thứ tự topo từ gốc tới lá ($A \to B, C \to D, E \to F, G$) luôn đảm bảo: với mỗi giá trị hợp lệ chọn cho nút cha, luôn tồn tại ít nhất một giá trị hợp lệ cho nút con kề nó. Do đó không bao giờ dẫn tới ngõ cụt $\implies$ 0 backtrack.

g) Xét theo từng cặp bước chẵn:

1. Ban đầu: AC được chạy trên toàn cây.
2. Gán $G = v_1$ (bước 1 lẻ, không chạy AC).
3. Gán $E = v_2$ (bước 2 chẵn, chạy AC): Nhờ bước 2 chạy AC, nếu lựa chọn tại G và E không tương thích với phần còn lại của đồ thị, AC sẽ phát hiện ngay miền rỗng và buộc E hoặc G đổi giá trị. Xấu nhất tại cặp này cần thử d giá trị của G, mỗi lần thử mất d giá trị của $E \implies$ backtrack tối đa cục bộ $O(d)$.
4. Sau khi cụm nhánh dưới tương thích, gán C, rồi gán A (bước 4 chẵn → chạy AC).
5. Tiếp tục gán B, rồi D (bước 6 chẵn → chạy AC).
6. Gán F (bước 7).

Trường hợp xấu nhất xảy ra do đi ngược từ lá lên gốc ở nhánh dưới ($G \to E \to C \to A$): do gán ở biến lẻ (G), lựa chọn có thể không báo trước được mâu thuẫn cho tới khi AC ở bước chẵn phát hiện $\implies$ số lần backtrack xấu nhất bị chặn trên bởi $O(d^2)$.

h) Thứ tự duyệt xen kẽ giữa hai nhánh lá độc lập (G ở nhánh dưới, F ở nhánh trên, rồi E, D...). Tại bước 1 gán G, bước 2 gán $F \implies$ chạy AC: Do G và F thuộc hai nhánh hoàn toàn độc lập nối qua A, việc kiểm tra AC giữa G và F không giúp phát hiện sớm mâu thuẫn ở các biến ở giữa (A, B, C).
Quá trình gán ngược từ các biến lá về gốc làm mất đi hoàn toàn lợi thế định hướng của cây. Trường hợp xấu nhất thuật toán phải duyệt vét cạn trên các miền giá trị giữa các cặp bước ⇒ Số lần backtrack trong trường hợp xấu nhất có thể lên tới cấp số nhân: $O(d^k)$ (với k là số tầng, tối đa $O(d^3)$ đến $O(d^4)$).

Bài 5:

1. Cụm nhánh ngoài cùng bên phải (đã cho mẫu trên hình để làm chuẩn quy luật):
   - Nút Casper trên hai lá $(3, 3)$ và $(9, 9)$:
     + $u = \min(3, 9) = 3$
     + v = max(3, 9) = 9 ⇒ Cặp $(3, 9)$ (hoàn toàn khớp với hình đề bài!).
   - Nút Casper trên hai lá $(8, 8)$ và $(4, 4)$:
     + $u = \min(8, 4) = 4$
     + v = max(8, 4) = 8 ⇒ Cặp $(4, 8)$ (hoàn toàn khớp với hình đề bài!).
   - Nút Pacman cha của $(3, 9)$ và $(4, 8)$:
     + Pacman chọn nhánh tối đa hóa u: $\max(3, 4) = 4$ (chọn nhánh phải).
     + Do đó: $u = 4$, và v lấy theo nhánh được chọn là $8 \implies$ Cặp $(4, 8)$.
2. Cụm dưới cùng bên trái:
   - Hai nút Casper ở tầng dưới cùng bên trái:
     + Nút Casper trên $(4, 4)$ và $(6, 6)$: đề bài đã điền sẵn $(4, 6)$.
     + Nút Casper trên $(1, 1)$ và $(7, 7)$: đề bài đã điền sẵn $(1, 7)$.
   - Nút Pacman cha của $(4, 6)$ và $(1, 7)$:
     + Pacman tối đa hóa u: so sánh $u_1 = 4$ và $u_2 = 1$ ⇒ chọn nhánh trái (vì 4>1).
     + Giá trị: $u = 4$, v lấy theo nhánh trái là 6 ⇒ Điền $(4, 6)$.
   - Nút Casper cha của lá $(2, 2)$ và nút vừa tính $(4, 6)$:
     + $u = \min(2, 4) = 2$.
     + Casper chọn nhánh có v lớn nhất: $v = \max(2, 6) = 6$.
     + Điền $(2, 6)$.
3. Cụm ở giữa:
   - Hai nút Casper tầng sâu nhất ở giữa:
     + Nút trên $(6, 6)$ và $(0, 0)$: đề bài đã cho $(0, 6)$.
     + Nút trên $(3, 3)$ và $(5, 5)$: đề bài đã cho $(3, 5)$.
   - Nút Pacman cha của $(0, 6)$ và $(3, 5)$:
     + Pacman so sánh u: 0 vs 3 ⇒ chọn nhánh phải ($3 > 0$).
     + Giá trị: $u = 3$, v lấy theo nhánh phải là $5 ⇒ Điền $(3, 5)$.
   - Nút Casper cha của lá $(1, 1)$ và nút vừa tính $(3, 5)$:
     + $u = \min(1, 3) = 1$.
     + Casper chọn nhánh tối đa hóa v: $v = \max(1, 5) = 5$.
     + Điền $(1, 5)$.
4. Nút Pacman tầng 2 (cha của cụm trái và cụm giữa):
   - Nhánh con trái có giá trị: $(2, 6)$.
   - Nhánh con phải có giá trị: $(1, 5)$.
   - Pacman tối đa hóa niềm tin u: so sánh $u_{trái} = 2$ và $u_{phải} = 1$ ⇒ Pacman chọn nhánh trái.

- Điền $(2, 6)$.

5. Nút Gốc (Pacman ở tầng cao nhất):
   - Con bên trái có giá trị: $(2, 6)$.
   - Con bên phải có giá trị: $(4, 8)$.
   - Pacman so sánh u: so sánh $u = 2$ (nhánh trái) và $u = 4$ (nhánh phải).
   - Pacman quyết định chọn nhánh phải (vì $4 > 2$).
   - Giá trị tại gốc: $u = 4$, v = 8 ⇒ Điền $(4, 8)$.

b)

Khi Casper tính toán giá trị thực tế dựa trên phân tích Alpha-Beta, nếu một nhánh con của Pacman đã có giá trị niềm tin u thấp hơn một nhánh khác chắc chắn được chọn, thì hành vi của Pacman là tất định (deterministic).

Tuy nhiên, vì quyết định của Pacman phụ thuộc vào u, còn quyết định của Casper phụ thuộc vào v (hai hàm mục tiêu không trùng khớp), việc cắt tỉa alpha-beta thông thường không thể áp dụng trực tiếp nếu không biết quan hệ phụ thuộc giữa u và v.

Nhánh duy nhất có thể bỏ qua là các nhánh lá phía sau một nút khi mà giá trị của nó không thể làm thay đổi lựa chọn argmax/argmin cục bộ của nút cha trực tiếp.

c)

Nếu Pacman biết Casper thân thiện, cả Pacman và Casper đều có chung hàm mục tiêu là MAX hóa tiện ích (trò chơi trở thành trò chơi hợp tác hoàn toàn - Fully Cooperative Game).

Khi đó, toàn bộ cây trở thành cây MAX-MAX (mọi nút đều chọn giá trị lớn nhất).

Giá trị toàn bộ cây sẽ là giá trị của chiếc lá lớn nhất có thể đạt được trên cây:

$V^* = \max(\textit{tất cả các lá}) = 9$

So sánh: Ban đầu trò chơi kết thúc với giá trị thực tế là 8. Khi Pacman biết Casper thân thiện, giá trị trò chơi tăng lên thành 9 (Pacman đi sang nhánh phải và đạt đến lá $(9, 9)$).

Bài 6:

Thuật toán	**Điều kiện đầy đủ**	**Điều kiện tối ưu**
Breadth-First Search (BFS)	- Hệ số rẽ nhánh hữu hạn ($b < \infty$). - Trên đồ thị: cần kiểm tra trạng thái trùng lặp (Graph Search).	Tất cả chi phí bước đi (hành động) là như nhau / bằng hằng số (unit step cost).
Depth-First Search (DFS)	- Không gian trạng thái hữu hạn VÀ phải là Graph Search (loại bỏ chu trình).	Không bao giờ đảm bảo tối ưu (luôn có thể trả về đường đi rất dài trước khi xét đường đi ngắn).

Iterative Deepening Search (IDS)	Hệ số rẽ nhánh hữu hạn ($b < \infty$).	Mọi chi phí hành động là như nhau (unit cost).
A* Search	- Chi phí mỗi bước đi bị chặn dưới bởi số dương: $c(s, a, s') \geq \epsilon > 0$. - Hệ số rẽ nhánh $b < \infty$.	- Tree Search: Heuristic $h(n)$ phải Admissible (không bao giờ đánh giá cao hơn thực tế). - Graph Search: Heuristic $h(n)$ phải Consistent (thỏa mãn bất đẳng thức tam giác).

Giải thích:

1. Điều kiện chi phí bước $\epsilon > 0$: Nếu tồn tại các cạnh có chi phí bằng 0 hoặc chu trình có tổng chi phí bằng 0, các thuật toán tối ưu hóa chi phí (như UCS, A^*) có thể bị mắc kẹt vô hạn trong việc duyệt các bước đi 0 đồng mà không bao giờ tiến tới đích (vi phạm tính đầy đủ).
2. Đặc tính Heuristic:

- Nếu $h(n)$ không admissible, A^* có thể đánh giá quá cao một đường đi tối ưu, khiến đường đi tối ưu bị xếp sau một đường đi kém hơn trong Priority Queue $\implies$ mất tính tối ưu.
- Trên đồ thị, nếu $h(n)$ admissible nhưng không consistent, một trạng thái có thể được lấy ra khỏi danh sách mở (expanded) lần đầu qua một con đường chưa phải tối ưu; nếu Graph Search không cho phép mở lại (re-open) trạng thái đã duyệt, A^* sẽ trả về nghiệm sai.

Bài 7:

a) Có thể thay đổi.

Chi phí của chính đường đi cũ chắc chắn tăng thành $C^* + n$.

Do đó, có thể xuất hiện một đường đi dự phòng khác trước đây có chi phí $C' > C^*$ nhưng hiện tại $C' < C^* + n$.

Cách cập nhật: Tăng $g(Y) = g(X) + c_{mới}$. Lan truyền cập nhật g-value xuôi theo các nút con cháu của Y trên cây tìm kiếm cũ. Đưa các nút bị ảnh hưởng vào lại Priority Queue và tiếp tục chạy A^* cho đến khi tìm được đích.

b) Chắc chắn đổi.

Chi phí tối ưu mới giảm chính xác còn $C^*_{mới} = C^* - n$.

Đường đi tối ưu mới: Vẫn chính là đường đi cũ! Vì đường đi cũ vốn đã là ngắn nhất, khi một

cạnh trên nó giảm chi phí mà các đường khác giữ nguyên thì nó lại càng tối ưu tuyệt đối.
Cách cập nhật: Không cần tìm kiếm lại! Chỉ cần trừ n vào tổng chi phí đường đi.

c) Hoàn toàn KHÔNG thay đổi.
Giải thích: Cạnh này vốn dĩ đã quá tốn kém nên không được chọn vào đường tối ưu cũ; nay chi phí của nó lại càng đắt thêm, nên nó chắc chắn không thể trở thành tối ưu.
Cách cập nhật: Không cần làm gì cả.

d) Có thể thay đổi.
Nếu chi phí đường đi qua cạnh này sau khi giảm: $g(X) + (c - n) + h(Y) < C^*$, thì cạnh này có khả năng tạo ra đường đi mới tốt hơn đường đi cũ.
Cách cập nhật: Cập nhật lại $g(Y) = \min(g(Y), g(X) + c_{mới})$. Đưa Y vào Priority Queue và tiếp tục cho A^* chạy tiếp từ trạng thái frontier hiện tại mà không cần reset đồ thị.

e) Hoàn toàn KHÔNG thay đổi.
Giải thích: Ngay cả với chi phí thấp hơn ban đầu mà thuật toán còn chưa cần đụng tới cạnh này trước khi kết thúc ở đích, thì việc nó tăng giá càng khiến nó không bao giờ được chọn.
Cách cập nhật: Giữ nguyên kết quả cũ, không làm gì cả.

f) Có thể thay đổi.
Cạnh này trước đây chưa được duyệt tới vì độ ưu tiên thấp, nhưng việc giảm chi phí có thể làm giá trị f qua $X \to Y$ giảm xuống dưới ngưỡng C^*.
Cách cập nhật: Đánh giá lại $f(Y) = g(X) + (c - n) + h(Y)$. Nếu $f(Y) < C^*$, đưa Y vào Priority Queue và tiếp tục bước lặp mở rộng của A^*.

Bài 8:
a) Các lớp tác tử tồn tại hàm f hợp lý hoàn hảo: Lớp B và Lớp C.

b) Các lớp tác tử tồn tại hàm f hợp lý hoàn hảo: Chỉ Lớp C.

c) Các lớp tác tử tồn tại hàm f hợp lý hoàn hảo: Chỉ Lớp C.
d) Đúng
Ví dụ: Môi trường chỉ có duy nhất một hành động khả thi (ví dụ: tập hành động $A = \{NoOp\}$). Dù tác tử là A, B hay C, tác tử chỉ có thể chọn hành động duy nhất đó, do đó hiển nhiên hành động đó tối đa hóa hiệu suất kỳ vọng (hợp lý tuyệt đối).